

Decision Tree

QUESTION 1

What is a Decision Tree? Explain its structure (Root, Nodes, Leaves) with a real-life example.

ANSWER

A Decision Tree is a supervised machine learning algorithm used for classification and regression problems. It makes decisions by splitting data into smaller groups based on conditions, similar to how humans make decisions.

Questions/conditions are asked at each step
Based on answers, we move to the next step
Finally, we reach a decision (output)

Structure of a Decision Tree

1) Root Node

The topmost node of the tree.
It contains the first question or condition.
It splits the data into different groups.

Example:

"Is the person older than 18?"

2) Nodes (Decision/Internal Nodes)

These are the points where the tree asks questions.
Each node represents a condition based on a feature.

Example:

Age > 18?

Yes No

Income? Reject

Here, Income? is a decision node.

3) Branches

Branches are the paths connecting nodes.
They represent the possible answers/results of a condition.

Example:

Age > 18?

|

Yes → Continue

No → Stop

4) Leaf Nodes

The final output/decision of the tree.
No further splitting happens here.

Example:

Loan Approval

Yes No

"Yes" and "No" are leaf nodes.

QUESTION 2	Differentiate between Gini Impurity and Entropy. Which one is used by default in Scikit-learn and why?
ANSWER	Both Gini Impurity and Entropy are measures used by a Decision Tree to decide the best split of data. They measure how mixed or impure the classes are in a node. Difference between Gini Impurity and Entropy Both Gini Impurity and Entropy are measures used by a Decision Tree to decide the best split of data. They measure how mixed or impure the classes are in a node. 1) Gini Impurity Gini Impurity measures the probability of incorrectly classifying a randomly chosen data point if it was labeled according to the class distribution. Formula: $Gini = 1 - \sum p_i^2$ Where: 2) Entropy Entropy measures the amount of uncertainty or randomness in the data. Formula: $Entropy = -\sum p_i \log_2(p_i)$ Example: Same data: 5 Yes 5 No Entropy = $-(0.5 \log_2(0.5) + 0.5 \log_2(0.5))$ Entropy = 1

QUESTION 3	What is Overfitting in Decision Trees? How can we detect it using training and testing accuracy?
ANSWER	Overfitting happens when a Decision Tree learns the training data too well, including noise and unnecessary details, instead of learning general patterns. The model performs very well on training data but performs poorly on new/unseen testing data. Suppose we train a Decision Tree to predict whether a student will pass. Training data: Age = 20 Study Hours = 5 Attendance = 90% Result = Pass The tree may create very specific rules: IF Age = 20 AND Study Hours = 5 AND Attendance = 90% THEN Pass This rule works for training data but may fail for new students. How to Detect Overfitting Using Training and Testing Accuracy? We compare: Training Accuracy Accuracy on data used to train the model. Testing Accuracy Accuracy on unseen data.

QUESTION 4	Explain Pruning in Decision Trees			
	What is the difference between Pre-pruning and Post-pruning?			
ANSWER				
	Pruning is a technique used to reduce the size and complexity of a Decision Tree by removing unnecessary branches.			
	The main purpose of pruning is to prevent overfitting and improve the model's performance on unseen data.			
	A fully grown Decision Tree may learn noise from training data:			
	1) Pre-pruning (Early Stopping)			
	Pre-pruning stops the tree from growing too much during training.			
	The tree growth is controlled by setting limits before the tree is created.			
	Common parameters:			
	a) max_depth			
	Limits the maximum depth of the tree.			
	b) min_samples_split			
	Minimum number of samples required to split a node.			
	c) min_samples_leaf			
	Minimum samples required in a leaf node.			

		2) Post-pruning (Late Pruning)						
		Post-pruning allows the tree to grow completely first, then removes unnecessary branches afterward.						
		Steps:						
		Build a full Decision Tree						
		Check unnecessary branches						
		Remove branches that do not improve performance						
		usually gives better accuracy						
		Finds the best tree size after seeing the full tree						

QUESTION 5	What is Feature Importance? How can it help businesses in decision-making?								
ANSWER									
	Feature Importance is a technique in Machine Learning that tells us which input features (variables) have the most impact on the model's prediction.								
	In a Decision Tree, feature importance shows how much each feature helps in making decisions while splitting the data.								
	How Feature Importance Helps Businesses in Decision-Making								
	1) Identifying Key Factors								
	Businesses can understand what factors drive outcomes.								
	Example:								
	A company analyzes customer churn.								
	Feature Importance:								
	2) Better Resource Allocation								
	Businesses can focus money and effort on important areas.								
	Example:								
	A marketing company finds:								
	3) Improve Decision Strategies								
	Feature importance helps managers make data-driven decisions instead of assumptions.								
	Example:								
	A hospital predicts disease risk.								
	Important features:								
	4) Remove Unnecessary Features								
	Low-importance features can be removed.								

QUESTION 6

Data Understanding

Load the dataset

Display first 5 rows

Check:

Shape

Data types

Column names

ANSWER

```
Home x bank.csv x Untitled36 +
localhost:8888/notebooks/Untitled36.ipynb?
[4]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

[7]: df = pd.read_csv('bank.csv')
df.head(), df.shape, df.dtypes, df.columns

[7]: (   age   job  marital  education  default  balance  housing  loan  contact \
0  59  admin.   married  secondary    no    2343     yes   no  unknown
1  56  admin.   married  secondary    no     45     no   no  unknown
2  41  technician  married  secondary    no    1270     yes  no  unknown
3  55  services  married  secondary    no    2476     yes  no  unknown
4  54  admin.   married  tertiary    no     184     no   no  unknown

   day month  duration  campaign  pdays  previous  poutcome  deposit
0    5   may    1042         1    -1         0  unknown    yes
1    5   may    1467         1    -1         0  unknown    yes
2    5   may    1389         1    -1         0  unknown    yes
3    5   may     579         1    -1         0  unknown    yes
4    5   may     673         2    -1         0  unknown    yes ,
(11162, 17),
age          int64
job          object
marital      object
education    object
default      object
balance      int64
housing      object
loan         object
contact      object
day          int64
month        object
duration     int64
campaign     int64
pdays      int64
previous     int64
poutcome     object
deposit      object
dtype: object,
Index(['age', 'job', 'marital', 'education', 'default', 'balance', 'housing',
       'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays',
       'previous', 'poutcome', 'deposit'],
      dtype='object'))
```

QUESTION 7

Data Cleaning

Check for missing values
Handle missing values (if any)
Check for duplicates
Interpret:
Is the dataset clean? What issues did you find?

ANSWER

```
[8]: df.isnull().sum()

[8]: age      0
     job      0
     marital  0
     education 0
     default  0
     balance  0
     housing  0
     loan     0
     contact  0
     day      0
     month    0
     duration 0
     campaign 0
     pdays    0
     previous 0
     poutcome 0
     deposit  0
     dtype: int64

[9]: df.duplicated().sum()

[9]: np.int64(0)
```

Data types: Correct
Dataset is ready for analysis/model building

QUESTION 8		Data Preprocessing					
		Convert categorical variables into numerical					
		Mention which encoding method you used					
		Interpret:					
		Why is encoding necessary for Decision Trees?					

ANSWER

The screenshot shows a Jupyter Notebook with the following code and output:

```
encoder = LabelEncoder()

#df['education'] = encoder.fit_transform(df['education'])
#df['deposit'] = encoder.fit_transform(df['deposit'])
#df.education,df.deposit
df= df[['education','deposit']]
df
```

The output of the code is a DataFrame with 11162 rows and 2 columns:

	education	deposit
0	1	1
1	1	1
2	1	1
3	1	1
4	2	1
...	...	...
11157	0	0
11158	1	0
11159	1	0
11160	1	0
11161	1	0

11162 rows × 2 columns

QUESTION 9

. Feature Selection & Splitting

ANSWER

Separate:

Features (X)

Target (y)

Perform train-test split (80-20)

In this step, we separate the dataset into:

Features (X) → Input variables used to make predictions

Target (y) → Output variable we want to predict

Then we split the data into:

80% Training data

20% Testing data

Separate features and target

```
X = df.drop('Loan_Status', axis=1)
```

```
y = df['Loan_Status']
```

drop() removes the target column from features

axis=1 means remove a column

y contains only the target variable

check X and Y

```
print(X.head())
```

```
print(y.head())
```

Import train_test_split:

```
from sklearn.model_selection import train_test_split
```

```
train, X_test, y_train, y_test = train_test_split(
```

```
X,
```

```
y,
```

```
test_size=0.2,
```

```
random_state=42
```

```
)
```

```
test_size=0.2
```

Means:

80% → Training data

20% → Testing data

[illegible]

QUESTION 10	Model Building		
ANSWER	Train a Decision Tree Classifier		
	Use:		
	<pre>criterion='gini'</pre> <pre>max_depth=5</pre>		
	Interpret:		
	What does max_depth control?		

QUESTION 11	Model Evaluation				
ANSWER	Perform:				
	Accuracy score				
	Confusion Matrix				
	Classification Report				
	Interpret (MUST):				
	Is the model performing well?				
	Explain Precision & Recall				
	Any class imbalance issue?				
	1. Accuracy Score				
	from sklearn.metrics import accuracy_score				
	y_pred = model.predict(X_test)				
	print("Accuracy:", accuracy_score(y_test, y_pred))				
	Accuracy = proportion of correctly classified samples.				
	Good for balanced datasets, but misleading if classes are imbalanced.				
	2. Confusion Matrix				
	from sklearn.metrics import confusion_matrix				
	cm = confusion_matrix(y_test, y_pred)				
	print("Confusion Matrix:\n", cm)				
	Shows True Positives (TP), True Negatives (TN), False Positives (FP), False Negatives (FN).				

		Helps identify where the model is making mistakes.				
		3. Classification Report				
		from sklearn.metrics import classification_report				
		print(classification_report(y_test, y_pred))				
		Provides Precision, Recall, F1-score for each class.				
		Interpretation				
		Is the model performing well?				
		Check:				
		Good performance:				
		High accuracy				
		Precision and recall are balanced				
		Confusion matrix has more TP and TN values				

QUESTION 13		Pruning Experiment	
ANSWER		Train another model with:	
		max_depth=3	
		min_samples_split=20	
		Interpret:	
		Compare performance with previous model	
		Which one is better and why?	

QUESTION 14		feature Importance	
ANSWER		Extract top 5 important features	
		Plot feature importance graph	
		Interpret:	
		Which features influence prediction the most?	

		Business Insights		
QUESTION 15		Based on your model:		
ANSWER		Answer:		
		Which type of customers are more likely to say “yes”?		
		What strategy should a bank us		